

## Lista Avaliativa 2 - Respostas

1)

Um processo não pode passar diretamente do estado bloqueado para o estado de execução. A transição ocorre em duas etapas. Primeiro, o processo precisa sair do estado bloqueado e ir para o estado pronto. Isso acontece quando o evento externo pelo qual ele estava esperando finalmente ocorre. Por exemplo, a conclusão de uma operação de entrada/saída, a chegada de dados de um dispositivo ou a liberação de um recurso que estava sendo aguardado.

Uma vez no estado pronto, o processo está apto a executar, mas ainda aguarda a sua vez de usar a CPU. A segunda etapa ocorre quando o escalonador do sistema operacional seleciona este processo, dentre todos os que estão no estado pronto, para ser o próximo a receber o tempo de CPU. Somente após ser escolhido pelo escalonador é que o processo finalmente passa para o estado de execução.

2)

A entrada de um processo na tabela de processos armazena todo o estado e contexto necessários para que o sistema operacional possa gerenciar o processo e restaurar sua execução após uma interrupção ou suspensão. As principais informações, e seus motivos, são:

- **Gerenciamento do Processo:**

- **Registradores e Contador de Programa:** Armazenam o conteúdo dos registradores da CPU no momento da interrupção. O motivo é permitir que o processo continue sua execução exatamente do ponto onde parou, como se nada tivesse acontecido.
- **Estado do Processo:** Indica se o processo está em execução, pronto ou bloqueado. Essa informação é crucial para que o escalonador saiba quais processos estão aptos a receber tempo de CPU.
- **Prioridade e Parâmetros de Escalonamento:** Utilizados pelo algoritmo de escalonamento para decidir qual processo deve ser executado em seguida.

- **Gerenciamento de Memória:**

- **Ponteiros para os Segmentos de Memória:** Apontam para as áreas de memória que contêm o código, os dados e a pilha do processo. O motivo é permitir que o sistema localize e gerencie o espaço de endereçamento do processo.

- **Gerenciamento de Arquivos:**

- **Descritores de Arquivo e Diretórios:** Mantêm informações sobre os arquivos que o processo abriu e seu diretório de trabalho atual. Isso é necessário para gerenciar os recursos de E/S que o processo está utilizando e controlar o acesso a eles.

3)

A diferença fundamental entre processos e threads está na forma como agrupam recursos e executam tarefas. Um processo é uma unidade de alocação de recursos, funcionando como um contêiner que agrupa

um conjunto de recursos relacionados. Por outro lado, uma thread é a unidade de execução, representando um fluxo de controle que executa o código dentro do ambiente de um processo.

Os itens que são únicos por processo e, portanto, compartilhados por todas as threads dentro dele, são:

- **Espaço de endereçamento:** Todas as threads compartilham a mesma visão da memória, incluindo o código e as variáveis globais.
- **Arquivos abertos:** Se uma thread abre um arquivo, as outras threads do mesmo processo podem acessá-lo.
- **Processos filhos e alarmes:** Recursos como estes pertencem ao processo como um todo.

Os itens que são únicos por thread, representando seu contexto de execução individual, são:

- **Contador de Programa (PC):** Indica qual instrução a thread está executando.
- **Registradores:** Armazenam as variáveis de trabalho atuais da thread.
- **Pilha de Execução (Stack):** Contém o histórico de chamadas de função e as variáveis locais de cada thread.
- **Estado:** Cada thread possui seu próprio estado (execução, pronto ou bloqueado).

#### 4)

**Threads no Espaço do Usuário:** Neste modelo, as threads são implementadas por uma biblioteca de tempo de execução no espaço de usuário, e o kernel do sistema operacional não tem conhecimento de sua existência. Para o kernel, o processo inteiro é uma única thread de execução.

- **Vantagens:**

- **Rapidez:** A criação, destruição e o chaveamento de contexto entre threads são extremamente rápidos, pois não exigem uma chamada de sistema para o núcleo.
- **Portabilidade:** Um pacote de threads de usuário pode ser implementado em qualquer sistema operacional, mesmo naqueles que não oferecem suporte nativo a threads.
- **Customização:** Cada processo pode ter seu próprio algoritmo de escalonamento de threads, customizado para sua aplicação.

- **Desvantagens:**

- **Bloqueio do Processo:** Se uma thread realiza uma chamada de sistema bloqueante (ex: leitura de disco), o processo inteiro é bloqueado pelo kernel, impedindo que outras threads do mesmo processo executem.
- **Falta de Paralelismo Real:** Como o kernel enxerga apenas uma thread por processo, ele não pode escalonar as threads do mesmo processo para executar em múltiplos núcleos de CPU simultaneamente.

**Threads no Kernel:** Neste modelo, o kernel do sistema operacional é responsável por gerenciar todas as threads. Ele mantém uma tabela de threads para todo o sistema e realiza o escalonamento no nível do kernel.

- **Vantagens:**

- **Não Bloqueio do Processo:** Se uma thread realiza uma chamada de sistema bloqueante, o kernel pode escalonar outra thread do mesmo processo para executar.

- **Paralelismo Real:** O kernel pode escalonar diferentes threads de um mesmo processo para executar simultaneamente em diferentes núcleos de CPU, permitindo paralelismo verdadeiro.

- **Desvantagens:**

- **Lentidão:** A criação, destruição e o chaveamento de contexto são consideravelmente mais lentos, pois cada operação exige uma chamada de sistema e uma mudança de modo (usuário para núcleo).
- **Maior Sobrecarga:** As estruturas de dados para gerenciar cada thread são mantidas no núcleo, o que consome mais recursos do sistema.

5)

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <time.h>

#define NUM_FILHOS 4
#define NUM_THREADS 3

void *funcao_thread(void *arg)
{
    int tempo_aleatorio = (rand() % 9) + 2;
    printf("Sou a thread %lu. Vou dormir por %d segundos\n",
pthread_self(), tempo_aleatorio);
    sleep(tempo_aleatorio);
}

int main()
{
    time_t tempo_inicio, tempo_fim;
    pid_t filhos[NUM_FILHOS];
    int pipefd[2];
    pipe(pipefd);

    tempo_inicio = time(NULL);

    for (int i = 0; i < NUM_FILHOS; i++)
    {
        filhos[i] = fork();

        if (filhos[i] < 0)
        {
            perror("Erro ao criar o filho");
            exit(1);
        }
        else if (filhos[i] == 0)
        {

```

```

        srand(time(NULL) + getpid());
        close(pipefd[0]);
        printf("Sou o filho %d.\n\n\n", getpid());

        pthread_t threads[NUM_THREADS];
        for (int j = 0; j < NUM_THREADS; j++)
        {
            pthread_create(&threads[j], NULL, funcao_thread, NULL);
        }

        for (int j = 0; j < NUM_THREADS; j++)
        {
            pthread_join(threads[j], NULL);
        }

        pid_t meu_pid = getpid();
        write(pipefd[1], &meu_pid, sizeof(meu_pid));
        close(pipefd[1]);
        exit(0);
    }
}

close(pipefd[1]);
int buffer = 0;

for (int i = 0; i < NUM_FILHOS; i++)
{
    read(pipefd[0], &buffer, sizeof(buffer));
    if (i == 0)
    {
        printf("\n=====\nPIDs recebidos:\n");
    }
    printf("> %d\n", buffer);
    fflush(stdout);
}

close(pipefd[0]);

for (int i = 0; i < NUM_FILHOS; i++)
{
    waitpid(filhos[i], NULL, 0);
}

tempo_fim = time(NULL);
printf("\n\nTempo decorrido em segundos: %lu\n", tempo_fim -
tempo_inicio);

return 0;
}

```

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#define STB_IMAGE_IMPLEMENTATION
#include <stb/stb_image.h>
#define STB_IMAGE_WRITE_IMPLEMENTATION
#include <stb/stb_image_write.h>

// Estrutura para passar parâmetros para as threads
typedef struct {
    unsigned char *img;
    unsigned char *final;
    int w;
    int h;
    int c;
    int canal;
    int filtro[3][3];
} ThreadData;

// Função que cada thread executará
void *processar_canal(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    for (int j = 0; j < data->h; j++) {
        for (int i = 0; i < data->w; i++) {
            int novo = 0;

            // Aplica a convolução
            for (int ii = -1; ii < 2; ii++) {
                for (int jj = -1; jj < 2; jj++) {
                    int cordx = ii + i;
                    int cordy = jj + j;
                    int tmp = 0;

                    if (cordx >= 0 && cordx < data->w && cordy >= 0 &&
cordy < data->h) {
                        tmp = *(data->img + data->canal + (cordx * data->c
+ cordy * data->w * data->c));
                    }

                    novo += tmp * data->filtro[ii + 1][jj + 1];
                }
            }

            // Limita os valores entre 0 e 255
            if (novo < 0) novo = 0;
            if (novo > 255) novo = 255;

            *(data->final + data->canal + (i * data->c + j * data->w *
data->c)) = novo;
        }
    }
}

```

```

    return NULL;
}

int main(void) {
    int w, h, c;
    unsigned char *img = stbi_load("imagem.jpg", &w, &h, &c, 0);

    if (img == NULL) {
        printf("Nao foi possivel ler a imagem\n");
        exit(1);
    }

    unsigned char *final = malloc(w * h * c);
    if (final == NULL) {
        printf("Erro ao alocar memoria\n");
        stbi_image_free(img);
        exit(1);
    }

    int filtro[3][3] = {{-1, -1, -1}, {-1, 8, -1}, {-1, -1, -1}};

    // Cria arrays para threads e dados
    pthread_t *threads = malloc(c * sizeof(pthread_t));
    ThreadData *thread_data = malloc(c * sizeof(ThreadData));

    if (threads == NULL || thread_data == NULL) {
        printf("Erro ao alocar memoria para threads\n");
        free(final);
        stbi_image_free(img);
        exit(1);
    }

    // Cria uma thread para cada canal
    for (int canal = 0; canal < c; canal++) {
        thread_data[canal].img = img;
        thread_data[canal].final = final;
        thread_data[canal].w = w;
        thread_data[canal].h = h;
        thread_data[canal].c = c;
        thread_data[canal].canal = canal;

        // Copia o filtro para a estrutura
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                thread_data[canal].filtro[i][j] = filtro[i][j];
            }
        }

        if (pthread_create(&threads[canal], NULL, processar_canal,
&thread_data[canal]) != 0) {
            printf("Erro ao criar thread %d\n", canal);
            free(threads);
            free(thread_data);
        }
    }
}

```

```
        free(final);
        stbi_image_free(img);
        exit(1);
    }
}

// Aguarda todas as threads terminarem
for (int canal = 0; canal < c; canal++) {
    pthread_join(threads[canal], NULL);
}

// Salva a imagem processada
stbi_write_jpg("imagem2.jpg", w, h, c, final, 100);

// Libera recursos
free(threads);
free(thread_data);
stbi_image_free(img);
free(final);

printf("Processamento concluido com sucesso!\n");

return 0;
}
```